

Making Formal Methods More Relevant to Software Engineering Students via Automated Test Generation

Gene Fisher

Department of Computer Science
California Polytechnic State University
San Luis Obispo, CA 93407
gfisher@calpoly.edu

Corrigan Johnson

Department of Computer Science
California Polytechnic State University
San Luis Obispo, CA 93407
johnsoncorrigan@gmail.com

ABSTRACT

The use of formal methods in software engineering has been advocated for a long time, by a lot of people. Unfortunately, advocates of formal methods remain a distinct minority among software engineering educators, as well as industrial practitioners. A number of reasons have been cited for the lack of acceptance of formal methods. Popular among these reasons is that formal methods lack relevance and utility to the everyday work of software engineers.

This paper presents a tool intended to increase the practical value of formal methods for software engineering students. The tool, called “Spest”, generates unit testing code from a formal program specification. There have been other such tools developed in the past, but all have been difficult for us to use in our educational setting. With Spest, we hope to overcome some of the difficulties with a tool that is easy to use and which generates readable and extendible testing code. Initial results of using Spest in our classes are promising and we are planning continued development.

Keywords

Formal methods; software testing; automated test generation; software engineering education

1. INTRODUCTION

Simply put, a “formal method” is one based on formal mathematical principles. In general, formal methods are an integral part of computer science education. They are of course used in theoretical computer science classes, where mathematical concepts are at the center of the curricula. Formal methods are also used in many other courses where mathematically-based concepts can be applied to various aspects of the course subject matter. In software engineering courses, formal methods can be used in all phases of software development, to provide benefits that include:

- precise specification of software requirements
- precise definitions of design and implementation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.

ITiCSE '16, July 09-13, 2016, Arequipa, Peru

© 2016 ACM. ISBN 978-1-4503-4231-5/16/07...\$15.00

DOI: <http://dx.doi.org/10.1145/2899415.2899424>

- formal basis for generating program tests
- basis for formal verification of program correctness
- clearer understanding overall of what software is supposed to do and how it does it

Despite such benefits being compelling to formal method devotees, the methods remain a hard sell to students.

Resistance to formal methods is by no means unique to software engineering courses. Computer science students often have difficulty in courses that employ formal mathematical reasoning. A number of educational studies have confirmed this. In [22] for example, Pillay puts it bluntly and succinctly in saying that students find mathematically-oriented course content “boring and difficult”.

We have observed the same student opinions in our software engineering courses. Furthermore, in addition to finding formal methods boring and difficult, our students also find them irrelevant to the software development work they need to do. Given such dismal student opinions, those of us who seek broader acceptance of formal methods clearly have our work cut out for us.

1.1 Our Experience with Formal Methods

For nearly three decades, we have tried a variety of approaches to teach formal methods in software engineering courses. We have prepared detailed course materials on the use of formal methods. We have designed course assignments to integrate formal methods into the work that students do. We have continually updated the tools we use for formal methods support, to keep them as up to date as possible. In graduate courses, we have had students read the literature on formal methods usage, including practical applications [20], educational benefits [23], and the “myth buster” papers that debunk popular misconceptions about formal methods [11, 4, 14]. For undergraduates, we have discussed the ideas presented in the literature and provided detailed examples for them to use in their course work.

Despite our best efforts, what students have taken away from our courses is that formal methods may have some *potential* benefits, but this potential remains unmet in the day-to-day work they do to engineer software. Outside of academia, it appears that many industrial practitioners have reached similar conclusions [25]. As educators, the industrial view does indeed figure into our work. This is because one of the factors cited for lack of industrial acceptance of formal methods is that students are not well trained in the methods. Hence, if we can succeed in improving student acceptance and use of formal methods, that may well have a positive effect on industrial use.

1.2 An Opportunity

In response to our lack of success, we have searched continuously for tools and techniques that could make formal methods of direct tangible benefit to our students. One particular technology has stood out in this regard – *automated generation of program tests from formal specifications*. There have been a number of tools developed for this purpose, but none has been realistically usable in our classes.

In the absence of a usable test generation tool, we have told our students of the promising research in this area, and said we have high expectations that such tools should soon be available. Alas, we have been waiting twenty years for this to happen. In the meantime, we have asked the following question in many student surveys: “*Suppose there were a tool that could generate usable tests from a formal software specification, alleviating a good deal of manual test implementation. Would the existence of such a tool make it worthwhile for you to write the specifications?*” The answer to this question has consistently been a clear “yes”.

In our experience, we have observed that our software engineering students believe these things:

- software testing is necessary, but writing tests by hand is tedious, boring, and difficult to do thoroughly
- formal software specification is unnecessary, but we could change our mind about this if there were a tool to generate genuinely usable tests from the specifications

We see from the second observation a clear opportunity to improve acceptance of formal methods. The test-generation system we are developing, named *Spest*, is designed to address this opportunity. Spest has two components:

- a simple specification language, based on well-established concepts of behavioral program specification
- a test generation component, that produces readable testing code, using testing techniques that are part of the software engineering course curriculum

2. BACKGROUND AND RELATED WORK

2.1 Specification Languages

A formal specification language is used to express abstractly the desired behavior of software. For the interested reader, an excellent in-depth survey of behavioral specification languages is provided by Hatcliff et al in [12].

Throughout the long history of formal specification, the key constructs have remained surprisingly stable. A widely used form of specification is based on *preconditions* and *postconditions*, dating back to the seminal 1967 paper by Robert Floyd [8]. This style of specification is reasonably well known in computer science, even appearing in a number of introductory computer science textbooks e.g., [13], as well as the 2013 ACM curriculum guidelines [15].

In an elementary form, preconditions and postconditions can appear as English-language comments associated with program functions, defining respectively what must be true before and after the function runs. In fully formal specification languages, preconditions and postconditions are compilable logic expressions, used for a number of purposes including runtime checks, as formulae for test-case generation, and as the basis of formal program verification.

Our earliest use of preconditions and postconditions was in the form of natural language program comments. Our

use of compilable specification languages began in the late 1980s with Larch [10] and Z [24]. In promoting the use of Larch and Z to students, we described them as a means to understand fully and unambiguously what a program does. While the students had some begrudging appreciation for why this is important, there were at least two significant practical problems with these languages:

1. their syntax and semantics were entirely separate from the programming languages used to implement the specifications
2. the tools available for the languages allowed the specifications to be checked for syntactic and type correctness, but the compiled specifications could be put to no other useful purpose by the students

An early language that addressed both of these problems was ADL – the Assertion Definition Language [5]. The ADL language itself was tailored for use with C programs. More importantly, the ADL environment provided one of the earliest practical tools to generate program tests from specifications. We discuss further details of the ADL test-generation strategy in the next subsection of the paper.

As promising as ADL was, it suffered from a somewhat common problem for formal specification languages – it went extinct. Even though ADL was backed by Sun Microsystems in the form of “SunTest”, it was a hard sell to a math-phobic software engineering community. Compounding the problem was the fact that software testing was also a bit of a hard sell to developers, in the pre-JUnit era.

To address the issues of language stability and lack of tools, we had a go at developing our own requirements specification language (RSL), initially under contract to NASA [9], and then extended for use in our software engineering courses. While the RSL toolset proved useful, the language still had the problem of being syntactically and semantically distinct from any implementation language.

With the advent of Java as a widely-used instructional language, an evolutionary step in formal specification was the introduction of several Java-specific notations. These include iContract, J@va, JASS, jContractor, and JML [17, 3, 1, 16, 18]. These notations were implemented using Java annotations and solved the problem of RSL and earlier languages being divorced from the concrete implementation language. Of these particular Java-based notations, all but JML have gone effectively extinct.

JML is alive and well; we had used it in our classes until last year, when we began to use Spest. JML has much to recommend it, including very active user and developer communities, as well as a number of useful tools. Further discussion of the JML test generation tools follows in the next subsection, followed after that with a discussion of why we have moved from JML to Spest.

While we have never used the Eiffel programming language [21] in any of our classes, it is certainly worth noting in this discussion of related work. Eiffel is one of the few procedural programming languages in which formal specification constructs are built in to the language. The design of Eiffel has clearly been influential, having been cited explicitly for example by the designers of iContract, JASS, jContractor, and JML. Also of note is that the Eiffel development environment includes functionality for test generation [19], about which we say more in the next subsection.

2.2 Test Generation from Specifications

As noted above, the ADL system was an early example of a practical tool for the automated generation of unit tests from formal specifications. The ADL test generator was based on the highly-influential research of Elaine Weyuker, who formulated the *meaningful impact* strategy for test generation [26]. The Spest test generator itself employs a variant of meaningful impact. A detailed technical discussion of this and other test generation techniques is beyond the scope of this paper. Suffice it to say that the early work of Weyuker and others made it clear that automated specification-based test generation can be practical and effective.

The test generation tools that are most directly related to Spest are those that have been developed for JML. These are JMLUnit [6] and JMLUnitNG [27]. JMLUnitNG was designed expressly as an upgrade to the earlier JMLUnit. With respect to Spest, one or both of these tools have the following significant shortcomings:

- an excessive number of tests are generated
- tests take an excessive amount of time to run
- generated test code is not as readable or extendible by students as it could be
- there is little to no active support for the tools

Among non-Java languages, the test generation tool provided in the Eiffel environment is definitely noteworthy [19]. It uses a different generation technique than ADL, JMLUnitNG, or Spest, but it does use formal program specifications. From a pedagogical standpoint, the goals for Eiffel test generation are in substantial alignment with our goals for Spest. In [19], the authors make the following observation, with which we completely agree:

“Our experience is that students who are less inclined towards mathematical techniques are more amenable to their study, use, and description if they are couched in terms of accepted engineering tasks, particularly testing, with which they have some experience.”

3. THE SPEST NOTATION

There are two over-arching goals for the Spest formal specification notation:

1. keep it as simple as possible
2. provide support for multiple programming languages

We have established these goals based on the reasons that we and others have observed for lack of acceptance of formal specification languages. We note also that these goals motivate our move from JML to Spest.

To achieve the first goal we have included in Spest a very small number of core constructs. To achieve the second goal, we have designed the core constructs so they can be readily incorporated into common programming languages.

The idea for the second goal stems from two separate but potentially conflicting language features. One feature is implementation-independent specification. While this can be desirable to allow flexibility in selecting an implementation language, we have noted its difficulties. Hence some have chosen the feature of a programming-language-specific specification notation. To support both these features, we have designed Spest so it can be incorporated into different “host” programming languages by absorbing as much as possible the standard syntax and semantics of that host.

```
abstract class UserDB {  
  
    Collection<UserRecord> data;  
  
    /** Add a new UserRecord to this.data.  
  
    pre:  
        // No existing record in the input data has  
        // the same ID as the new record.  
        !exists (UserRecord other_rec ;  
            data.contains(other_rec) ;  
            other_rec.id.equals(rec.id));  
  
    post:  
        // A record is in the output data if and  
        // only if it's the new record to be added  
        // or it's already in the input data.  
        forall (UserRecord any_rec ;  
            (data'.contains(any_rec)) iff  
            any_rec.equals(rec) ||  
            data.contains(any_rec));  
  
    */  
    abstract void add(UserRecord rec);  
}
```

Figure 1: A Spest example.

At present, we have implemented the Java version of Spest. We have near-term plans for C, C++, and Python versions. We believe this multi-language agenda is realistic given our previous experience developing multi-language programming environments [7] and the wide availability of open-source compilation tools.

Figure 1 shows an excerpt of a Spest specification. The example illustrates important Spest features:

- Spest keywords: **pre**, **post**, **forall**, **exists**, **iff**
- quantifier expressions
- the apostrophe character (read “prime”), used to distinguish pre versus post (primed) identifier values

Constructs not shown in Figure 1 are the **except** clause for exceptional behavior and the use of standard Java keywords **return**, **throw**, and **if-else** in expression contexts.

At first glance, one might say of Figure 1 that we have failed to meet our goal of notational simplicity. Indeed, the notation in Figure 1 is rather dense, which is one of the red flags raised in Wassyng’s critique of formal notations [25]. In our own defense, we can say that the example in Figure 1 was chosen purposely to show a non-trivial specification, with notation that is admittedly dense. In many cases, preconditions and postconditions are much simpler than this, particularly ones that do not use quantifiers.

Many students do reel at the notational density of examples like Figure 1. However, we have seen very similar perplexed reactions with other difficult subject matter, such as students’ first look at the density of a triple-reference operator in ANSI C (e.g., *****ptr**). The point is that students accept complexity when they see its ultimate benefits.

Pedagogically, we use common techniques to discuss the dense notation. We give the English transliteration and describe how it is used as inline comments. We tell students

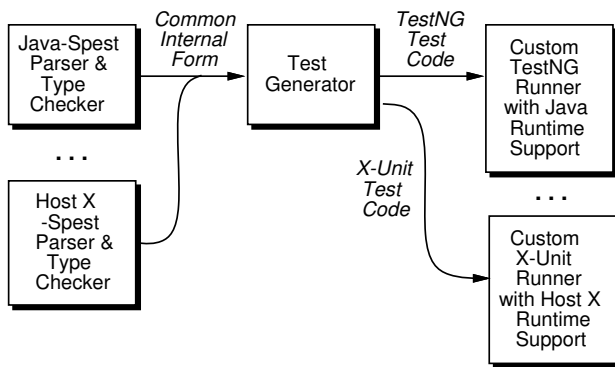


Figure 2: Components of the Spest system.

about reusable specification idioms. For example, the `add` method precondition uses an idiom for ensuring item uniqueness in a collection, and the postcondition uses an idiom for the precisely correct addition of an item into a collection. Such idioms can be used like design patterns, to characterize and categorize common idiomatic forms.

Our class materials also include detailed discussion of specification-based test generation techniques:

- provide input values that fully exercise the boolean logic of the preconditions, including data range conditions, collection-size ranges, and pairwise combinations of multiple input values
- use the postcondition as the testing oracle to define the expected output of the method
- use a coverage tool to measure test adequacy, writing additional white-box tests when necessary

Despite our best efforts to encourage use of these techniques, the testing practices actually employed by many of our students go something like this:

- do a half-hearted job of writing specifications
- use one's own familiarity with the code, together with an ad hoc application of black-box and white-box testing techniques to develop a substandard set of tests
- be satisfied with a low grade on project testing

Given these observations about students' testing practices, our goal for Spest is not to have students simply *accept* the utility of formal specification. Rather, the goal is to provide tool support that shows the concrete utility of specifications and help students achieve better testing results.

4. GENERATING TESTS WITH SPEST

Figure 2 is a high-level flow diagram of the Spest system. The first step in the test generation process is to parse and type check the specification. As new parser/type-checker components are added, they will produce the same internal form as currently produced by the Java-based component. This internal form is comprised of an abstract syntax tree and symbol table, sent to the test generation component.

The test generation component uses the same black-box test generation techniques we discuss in class. The generated testing code is in a readable form, so that students can (a)

```
public class UserDBTest {
    @Before
    public void setUp() {
        UserDB testObj = (UserDB)javaTestUtility.
            getSampleObject(classUnderTest);
    }
    @Test
    public void testAdd() throws Exception {
        // Clone pre-execution collection values
        java.util.Collection<UserRecord> data =
            cloner.deepClone(getFieldValues(...));

        // Generate basic test inputs
        List<UserRecord> testValues =
            javaTestUtility.getSampleObjects(...);

        // Generate inputs to exercise quantifiers
        List<UserRecord> quantTestValues =
            javaTestUtility.getUniversalValues(...);

        // Generate pairwise input combinations
        int[] [] combos =
            CombinationSupport.getCombinations(...);

        // Loop through each test case
        for (int i = 0; i < combos.length; i++) {

            // Invoke the method under test
            javaTestUtility.invokeMethod(...);

            // Validate method postcondition
            Assert.assertTrue(javaTestUtility.
                validatePostcondition(...));
        }
    }
}
```

Figure 3: Spest-generated testing code.

understand it as an example of applied testing techniques, and (b) extend it when it achieves inadequate test coverage. The generator can produce inadequate tests due either to an inadequate specification or the inherent limitations of its less-than-artificially intelligent algorithms.

The Spest system has a simple stand-alone GUI. It allows users to select code files to be tested, validate the specifications in that code, and generate the tests. The generated test code uses the TestNG library [2]. Users employ their preferred IDE to compile the tests with the application files under test. Part of the compilation process is the inclusion of a Spest runtime jar file. The pre-compiled runtime support has mnemonically-named methods that are called from the testing code. The support methods encapsulate lower-level aspects of test execution, so when students view the generated test code, higher-level testing concepts are more clearly evident. Also, the generated test code is clearly commented to indicate where students can add testing extensions.

Figure 3 has an excerpt of the testing class that Spest generates for the `UserDB` class shown in Figure 1. The excerpt is essentially as-generated, but with some elisions for brevity. It shows the major aspects of a generated test for a specification that uses unbounded quantification over a collection class, which is one of the more challenging aspects

Please rate your level of agreement with the following statements on a scale from 1 (disagree strongly) to 5 (agree strongly).

1. Using **L** for formal specification helped me understand the programs I developed, better than if **L** had not been used.
2. The **L** specifications written by my teammates helped me understand the intended behavior of their code.
3. When using **L**, I had to write additional support code that I would not have written if **L** had not been used.
4. Using **L** helped with the development of unit tests for the programs I wrote.
5. As a language, **L** was easy to understand.
6. The **L** compilation tools were easy to use.
7. The **L** documentation was useful.

Please describe your experience and any problems you encountered while using **L**. (Response here is optional free-form text.)

Figure 4: Survey questions.

of test generation. A detailed discussion of the generated testing code is beyond the scope of this paper. The inline comments explain the major functionality.

5. USING SPEST IN CLASSES

In the Fall and Winter quarters of the 2014-15 academic year, we used JML in a two-quarter software engineering sequence taken by junior and senior students. In the Winter and Spring quarters of that same academic year, we ran exactly the same course sequence using Spest. All course materials in the JML and Spest versions of the courses were the same, except for the use of JML versus Spest.

Our initial plan was to compare the test generation capabilities of JMLUnitNG and Spest. However, we were unable to use JMLUnitNG with student projects due to our use of Java 8, as well as other usage problems with JMLUnitNG. Given these problems, we chose to compare JML and Spest without having the students use the Spest test generator. Hence the studied comparison of JML and Spest is of their use as specification languages. We did perform a post hoc quantitative analysis, as described in Section 5.2.

5.1 Qualitative Student Survey

Figure 4 shows the survey questions we asked students at the end of the two-quarter sequences for JML and Spest. The letter “**L**” was substituted with “JML” or “Spest”.

Table 1 shows the statistical results of the survey. The survey population size was 25 students for the JML class and 22 for the Spest class. For all questions, the data show no statistically significant difference between JML and Spest. This is an important result for future usage studies. Namely, since we see no significant difference between using Spest and its best alternative, we can use Spest alone in further studies that assess the effectiveness of Spest’s specification-based test generation.

Overall, the results of our qualitative survey are consistent with many past surveys. That is, student attitudes are neutral at best towards formal methods in and of themselves, without something to make them more relevant.

5.2 Quantitative Analysis of Test Generation

As noted above, we were unable to compare tests gen-

Table 1: Statistical summary of survey results.

Ques:	1	2	3	4	5	6	7
JML:							
Mean	2.68	2.72	2.52	2.84	2.96	2.92	2.64
Std Dev	1.22	1.4	1.29	1.43	1.17	1.15	1.19
Spest:							
Mean	2.45	2.77	2.91	2.77	2.86	2.59	2.77
Std Dev	0.96	1.11	1.15	1.07	1.17	0.85	0.92
Stats:							
T-Test:	0.47	0.89	0.28	0.85	0.82	0.27	0.68
Sig?	No	No	No	No	No	No	No

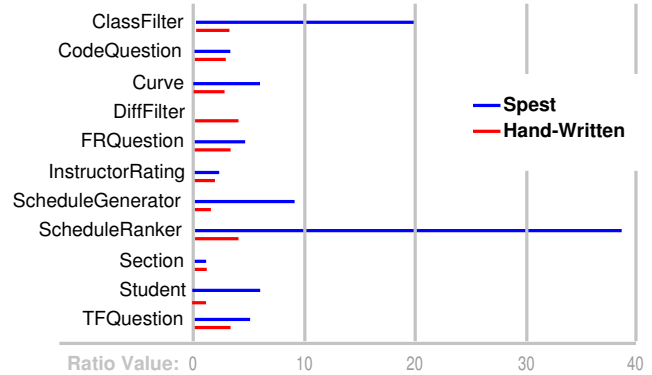


Figure 5: Efficiency of Spest vs. hand-written tests.

erated from JML versus Spest. The 2014-15 data do provide the basis for comparing hand-written tests to Spest-generated tests. Since the quality of Spest-generated tests is based on the quality of the specification, we chose student-written Java classes that met both of the following criteria:

1. they were well specified, based on human inspection
2. they were well tested, based on code coverage

For all classes that met these criteria, we generated Spest tests to compare to the student-written tests.

Given the observations made earlier about student testing practices, the number of student-written classes that met both of these criteria was relatively small. Furthermore, code coverage alone was not a useful distinguishing metric for hand-written versus Spest-generated, since coverage was high for both categories. Hence, we formulated a metric we believe provides a useful quantitative comparison.

The metric we used is a *test efficiency ratio*. This is defined as the ratio of line coverage to the number of lines of testing code to obtain that coverage. While this is a non-standard metric, it is a useful way to perform quantitative analysis of the data on hand. Figure 5 shows the results of the analysis. The left column in the figure has the names of Java classes that met the preceding two criteria. The red and blue lines compare efficiency ratios. The data show that Spest-generated tests have a higher efficiency ratio in all but one case, being statistically significant in four cases.

6. CONCLUSIONS AND FUTURE WORK

Our results so far are encouraging. For the 2015-16 academic year we are planning a controlled study using Spest

with and without automated test generation, in two large sections of software engineering.

We also have a large body of archived student projects from past software engineering courses. We plan to mechanically translate formal specifications written in past specification languages into Spest, generate tests from those specifications, and then compare the Spest-generated tests with student-written tests. We will use well-accepted test comparison metrics including code coverage and mutation.

Outside of classes, we plan to use Spest to generate tests for suitable open source projects that are well tested and for which specifications exist or could be feasibly written by us. Open-source projects using JML are good candidates. With hand-written and Spest-generated tests for the same code base, we will compare their quality using standard metrics.

In the way of tools, an Eclipse plug-in is also planned, to replace the stand-alone GUI.

7. REFERENCES

- [1] D. Bartetzko, C. Fischer, M. Möller, and H. Wehrheim. Jass – java with assertions. *Electronic Notes in Theoretical Comp. Sci.*, 55(2):103–117, 2001.
- [2] C. Beust and H. Suleiman. *Next Generation Java Testing: TestNG and Advanced Concepts*. Addison-Wesley Professional, 2007.
- [3] I. B. Bourdonov, A. V. Demakov, A. A. Jarov, A. Kossatchev, V. V. Kuliemin, A. Petrenko, and S. V. Zelenov. Java specification extension for automated test development. In *Revised Papers from the 4th Intl. Andrei Ershov Memorial Conference on Perspectives of System Informatics: Akademgorodok, Novosibirsk, Russia, PSI '02*, pages 301–307, London, UK, UK, 2001. Springer-Verlag.
- [4] J. P. Bowen and M. G. Hinchey. Seven more myths of formal methods. *IEEE Software*, 12(4):34–41, 1995.
- [5] J. Chang, D. J. Richardson, and S. Sankar. Structural specification-based testing with adl. In *Proc. of the 1996 ACM SIGSOFT Intl. Symposium on Software Testing and Analysis, ISSTA '96*, pages 62–70, New York, NY, USA, 1996. ACM.
- [6] Y. Cheon and G. T. Leavens. A simple and practical approach to unit testing: The jml and junit way. In *Proc. of the 16th European Conference on Object-Oriented Programming, ECOOP '02*, pages 231–255, London, UK, UK, 2002. Springer-Verlag.
- [7] G. Fisher. An overview of a graphical multilanguage applications environment. *IEEE Transactions on Software Engineering*, 14(6):774–786, 1988.
- [8] R. W. Floyd. Assigning meanings to programs. *Proc. of Symposium on Applied Mathematics*, 19:19–32, 1967.
- [9] D. Frincke, D. Wolber, G. Fisher, and G. C. Cohen. Requirements specification language (rsl) and supporting tools. NASA Contractor Report 189700, NASA Langley Research Center, 1992.
- [10] J. V. Guttag, J. J. Horning, and J. M. Wing. The larch family of specification languages. *IEEE Software*, 2(5):24–36, Sept 1985.
- [11] A. Hall. Seven myths of formal methods. *IEEE Software*, 12:11–19, 1990.
- [12] J. Hatcliff, G. T. Leavens, K. R. M. Leino, P. Müller, and M. Parkinson. Behavioral interface specification languages. *ACM Comput. Surv.*, 44(3):16:1–16:58, June 2012.
- [13] C. S. Horstmann. *Big Java*. Wiley Publishing, 4th edition, 2010.
- [14] C. Jaspan, M. Keeling, L. Maccherone, G. L. Zenarosa, and M. Shaw. Software mythbusters explore formal methods. *IEEE Software*, 26(6):60–63, 2009.
- [15] A. f. C. M. A. Joint Task Force on Computing Curricula and I. C. Society. *Computer Science Curricula 2013: Curriculum Guidelines for Undergraduate Degree Programs in Computer Science*. ACM, New York, NY, USA, 2013. 999133.
- [16] M. Karaorman and P. Abercrombie. jcontractor: Introducing design-by-contract to java using reflective bytecode instrumentation. *Formal Methods in System Design*, 27:275–312, 2005.
- [17] R. Kramer. icontract - the java(tm) design by contract(tm) tool. *Technology of Object-Oriented Languages, Intl. Conference on*, 0:295–308, 1998.
- [18] G. T. Leavens, A. L. Baker, and C. Ruby. Preliminary design of jml: A behavioral interface specification language for java. *SIGSOFT Softw. Eng. Notes*, 31(3):1–38, May 2006.
- [19] A. Leitner, I. Ciupa, M. Oriol, B. Meyer, and A. Fiva. Contract driven development = test driven development - writing test cases. In *Proc. of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering, ESEC-FSE '07*, pages 425–434, New York, NY, USA, 2007. ACM.
- [20] T. Margaria and B. Steffen, editors. *ISoLA '12: Proc. of the 5th Intl. Conference on Leveraging Applications of Formal Methods, Verification and Validation: Applications and Case Studies - Volume Part II*, Berlin, Heidelberg, 2012. Springer-Verlag.
- [21] B. Meyer. *Eiffel: the Language*. Prentice-Hall, 1st edition, 1992.
- [22] N. Pillay. Learning difficulties experienced by students in a course on formal languages and automata theory. *SIGCSE Bull.*, 41(4):48–52, Jan. 2010.
- [23] A. K. Sobel and M. Clarkson. Formal methods application: An empirical tale of software development. *IEEE Transactions on Software Engineering*, 28(3):308–320, 2002.
- [24] J. Spivey. *Understanding Z, A Specification Language and its Formal Semantics*. Cambridge University Press., 1st edition, 1998.
- [25] A. Wasssyng. Though this be madness, yet there is method in it? (keynote). In *1st FME Workshop on Formal Methods in Software Engineering (FormaliSE)*, ICSE '13, pages 1–7, Piscataway, NJ, USA, 2013. IEEE Press.
- [26] E. Weyuker, T. Goradia, and A. Singh. Automatically generating test data from a boolean specification. *IEEE Transactions on Software Engineering*, 20(5):353–363, 1994.
- [27] D. M. Zimmerman and R. Nagmoti. Jmlunit: The next generation. In *Proc. of the 2010 Intl. Conference on Formal Verification of Object-oriented Software, FoVeOOS'10*, pages 183–197, Berlin, Heidelberg, 2011. Springer-Verlag.